part03_cli — Development Environment and adk CLI

Run from the `part03_cli` directory (= parent directory of the agents). This corresponds to lecture notes **Part 3 · 03. Development Environment and CLI** (Slides 30–48).

```
part03_cli/
├── my_agent/              # 📁 Basic ADK agent scaffold (prompt-based Q&A)
├── my_agent_mod/          # 📁 Custom ADK agent (adds get_weather weather lookup to
├── my-agents-cli/         # 📁 Standard enterprise template from agents-cli (FastAP
├── my-agents-cli-mod/     # 📁 Custom agents-cli agent (real-time Open-Meteo weathe
├── run_direct_runner.py   # 🔑 Direct Runner code: register → run → consume events
├── adk_internals.py       # 🟢/🔑 Recreating the 5 steps that adk handles automati
├── adk_cli_map.py         # 🟢 Checking adk CLI source structure: command tree · Ag
├── api_client.py          # 🔑 Invoking /run and /run_sse of adk api_server (Slide
├── _path.py               # ⚙️ Parent directory and package path search helper
├── README-KR.md           # 📄 Lab guide in Korean
└── README-EN.md           # 📄 Lab guide in English (this file)
```

You only need one `.env` file at the lab root (`ADK_Labs/.env`). adk traverses up parent directories from the agent folder to locate it.

📂 Subdirectory Overview and Summary

Subdirectory	Tool / Framework	Core Details	Primary Run Commands
`my_agent`	adk CLI	The purest minimal ADK agent scaffolded via `adk create my_agent`. Answers general questions using Gemini's native LLM reasoning without external tools.	`adk run my_agent` `adk web --port 8000`
`my_agent_mod`	adk CLI	City weather lookup agent adding a local `get_weather` function tool (Function Calling) to the basic ADK scaffold.	`adk run my_agent_mod` `adk web --port 8000`

`my-agents-cli`	google-agents-cli	Enterprise-grade production template generated with `agents-cli create`. Includes FastAPI backend, SSE streaming, A2A protocol, Reasoning Engine adapter, automated tests (unit/integration/eval), and Terraform IaC.	`cd my-agents-cli`
			`agents-cli playground`
			`agents-cli install`

Subdirectory	Tool / Framework	Core Details	Primary Run Commands
`my-agents-cli-mod`	`google-agents-cli`	Enhanced agent building upon `my-agents-cli` by modifying `app/agent.py` to query real-time external Open-Meteo REST APIs and IANA timezones worldwide.	`cd my-agents-cli-mod` `agents-cli playground`

1) `my_agent/` Summary

- **Structure**: `agent.py`, `__init__.py`, `.env`, `.gitignore`, `.adk/session.db`
- **Role**: Minimal baseline ADK agent (`gemini-3.5-flash`). Executes default system prompt instructions without extra tools.
- **Run**: From `part03_cli`: `adk run my_agent` or `adk web --port 8000`

2) `my_agent_mod/` Summary

- **Structure**: `agent.py`, `__init__.py`, `.env`, `.gitignore`, `.adk/session.db`
- **Role**: Weather-specialized agent (`weather_agent`) with local tool `get_weather(city: str)`.
- **Run**: From `part03_cli`: `adk run my_agent_mod` or `adk web --port 8000`

3) `my-agents-cli/` Summary

- **Structure**: `app/` (FastAPI, agent, A2A, Reasoning Engine), `tests/` (unit, integration, eval), `deployment/terraform/`, `pyproject.toml`, `Dockerfile`, etc.
- **Role**: Standard enterprise template equipped with mock tools (`get_weather`, `get_current_time`) and production features (A2A protocol, evaluation pipeline, Terraform infrastructure deployment).
- **Run**: Inside `my-agents-cli/`: `agents-cli playground` or `uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000`

4) `my-agents-cli-mod/` Summary

- **Structure**: `my-agents-cli` architecture + `app/agent.py` real-world API integration + Web architecture guide slide deck
- **Role**: Real-time live weather and world clock queries via Open-Meteo API, complete with offline fallback (`AGENT_OFFLINE=1`).
- **Run**: Inside `my-agents-cli-mod/`: `agents-cli playground`

---

1. Same Agent, Two Execution Methods (Slides 23–25 · 31–39)

The backbone of an ADK application is the `Runner`. We run and compare the same `my_agent/agent.py` using two methods.

Method	What the Developer Writes	Hands-on Script
Direct Runner Code	root_agent + Runner · services · I/O handling code	`run_direct_runner.py`
adk CLI	root_agent (agent.py) only	`adk run my_agent` · `adk_internals.py`

① Direct Runner Code: Register → Run → Consume Events (🔑)

```
uv run run_direct_runner.py "Tell me the weather in Busan"
```

```
① Register   Runner(agent=weather_agent, session_service=InMemorySessionService)
② Execute    run_async(new_message='Tell me the weather in Busan')
   [Event 1] Model call → function_call get_weather({'city': 'Busan'})   (Model onl
   [Event 2] Tool execution → function_response get_weather: {'status': 'success',
③ Consume   Final Response (Event 3): The current weather in Busan is overcast with
```

A single `run_async()` corresponds to one full cycle of the agent loop (Slide 25): when the model requests a `function_call`, the Runner invokes the tool and calls the model again with the output.

② What adk Handles on Your Behalf (🟢 ①~④ without API key, 🔑 ⑤)

Recreates the 5 steps that `adk run my_agent` performs internally using ADK's native code (`AgentLoader`, `load_dotenv_for_agent`, `create_*_service_from_options`).

```
python .\adk_internals.py                                    # Default: recreate
python .\adk_internals.py --in_memory                        # Use in-memory serv
python .\adk_internals.py --session_service_uri sqlite:///tmp_sessions.db
python .\adk_internals.py --ask "Seoul weather?"              # Make an LLM call
```

By default, the script finds `my_agent`, loads `.env`, creates the session, artifact, and memory services, then builds a `Runner` and prints details for each step. Without `--ask`, it exits there: it does not wait in an interactive REPL, start a server, or call the LLM. The default run therefore does not require an LLM API key.

`--in_memory` selects in-memory services for sessions and artifacts. `--session_service_uri` lets you specify the session store directly. These options control the services configured by this script.

With `--ask "question"`, the script creates a new session and runs one turn by passing the question to `runner.run_async()`. It prints model responses and tool-call information. This makes a real LLM call and requires an API key and model configuration.

```
① Discover       AgentLoader → LlmAgent name='weather_agent' (from my_agent/agent.p
② Load .env      GOOGLE_API_KEY set=True · ADK_MODEL=gemini-3.5-flash-lite
③ Build Services session=PerAgentDatabaseSessionService → my_agent/.adk/session.db
                 artifact=PerAgentFileArtifactService · memory=InMemoryMemoryServic
④ Build Runner   Runner(app='my_agent', root_agent='weather_agent')
⑤ Provide Entry  In adk run, launches input('[user]: ') loop → runner.run_async()
```

- **The default session store is not InMemory.** Without extra flags, sessions are stored in the agent folder's `.adk/session.db` (SQLite). Specifying `--in_memory` or running in Cloud Run/Kubernetes defaults to InMemory. In production, use `--session_service_uri`.
- `adk web` and `adk api_server` build the exact same services and Runner; the only difference is the HTTP entry point ⑤ (Slide 38).
- `google.adk.cli.utils` is an internal ADK API that may change across versions (recreated here based on version 2.6.3 for learning).
- Running `adk run` or `adk_internals.py` creates the `my_agent/.adk/` folder, which can be safely deleted.

---

2. agents-cli (Slides 44–48, details in Part 13 Slides 143–156)

For an introduction to `agents-cli`, comparison with `adk`, command maps, project structures, and runtime mechanics, see `part13_agents_cli/`.

---

3. adk CLI Internal Structure (Slides 32–36)

Inspect the `google/adk/cli` source code (google-adk 2.14.0 = GitHub main) aligned with the lecture slides. Runs without an API key.

```
python adk_cli_map.py            # commands · loader · services · routes
```

Item	Slide	What to Check
`commands`	32	`adk` = `cli_tools_click.main` Click group, handler function per command
`loader`	34	`AgentLoader.load_agent('my_agent')` → `LlmAgent`
`services`	34	Session: `memory` · `sqlite` · `postgresql` · `mysql` · `agentengine`, Artifact: `memory` · `gs` · `file` …
`routes`	35–36	api_server routes: 22 · adk web routes: 59 (`/dev/*`: 35)

What `adk_cli_map.py` Does

```
python .\adk_cli_map.py                 # Inspect all four areas
python .\adk_cli_map.py commands        # Inspect only the command tree
python .\adk_cli_map.py loader          # Inspect only the agent loader
python .\adk_cli_map.py services        # Inspect only service URI schemes
python .\adk_cli_map.py routes          # Inspect only API routes
```

With no argument, the script behaves as if you passed `all` and prints these four sections in order:

- **commands**: Walks the installed ADK Click command group and prints the `adk` command tree and each command's handler.
- **loader**: Uses `AgentLoader` to find and load `my_agent` from `part03_cli`, then prints the loaded object's type and name and the agent search order. It loads the agent definition; it does not run a user query.
- **services**: Prints the URI schemes registered by ADK for session, artifact, and memory services.
- **routes**: Builds the FastAPI apps in memory for `api_server` (`web=False`) and `adk web` (`web=True`), then prints each app's route count, `/run*` routes, `/dev/*` count, and examples of web-only routes. It does not start a server or send an agent query.

This is an inspection tool for learning ADK internals. It does not make LLM calls and does not require an API key. Route counts and command listings may vary with the installed ADK version.

---

4. Key adk Commands (Slides 40–43)

Command	Description
`adk create my_agent2`	Scaffolds agent.py · __init__.py · .env
`adk run my_agent`	Terminal REPL (e.g., `Tell me Seoul weather`)
`adk run --save_session --session_id s1 my_agent`	Saves `my_agent/s1.session.json` on exit → reuse for evaluation
`adk run --resume my_agent/s1.session.json my_agent`	Resumes conversation from a saved session
`adk run --replay queries.json my_agent`	Executes inputs from `{"state": {...}, "queries": [...]}` without manual input (not a session file)
`adk web --port 8000`	Developer Web UI: http://localhost:8000 (Events · Trace · State · Graph tabs)

Command	Description
`adk api_server --port 8000 --session_service_uri="sqlite:///sessions.db"`	Headless REST/SSE server

Flags may differ across versions; check `adk run --help` for the latest options.

---

5. Calling api_server

When you start `adk api_server` from `part03_cli`, it serves the agents in its subdirectories through the API. Starting the server does not run one particular agent; the `app_name` in each `/run` or `/run_sse` request selects the agent. Check the available agents with `GET http://localhost:8000/list-apps`.

```
adk api_server --port 8000
```

In another terminal:

```
uv run api_client.py
```

`api_client.py` defaults to `APP = "my_agent"`. Change it to `APP = "my_agent_mod"` to call the agent that includes the weather tool. For curl requests, set the agent name in both `app_name` and the session URL.

`/run` vs. `/run_sse`

Both endpoints send a query to the same agent; they differ in how events are delivered.

Endpoint	Response behavior	Client usage
`POST /run`	Returns the complete set of events as a JSON array after the agent finishes	Process the array after the request completes
`POST /run_sse`	Streams events as Server-Sent Events (SSE) as they are generated	Keep the connection open and read/process events incrementally

Use `/run` when you want the complete result at once, and `/run_sse` when you want to consume events or show progress while the agent is running. In `api_client.py`, the first `/run` request collects all events for `Seoul weather?`; then `/run_sse` sends `What about Busan?` in the same session and prints each event as it arrives. These are separate agent turns.

Calling directly with curl:

```
# Create session
curl -X POST localhost:8000/apps/my_agent_mod/users/u1/sessions/s1

# Query agent
curl -X POST localhost:8000/run -H "Content-Type: application/json" -d "{\"app_name
```